LRU Cache

evict the entry that was used least recently

Watch the 4-min video

In one line: LRU keeps recently used entries and evicts the one untouched for longest; it's built with a hash map from key to node plus a doubly linked list ordered by recency, so get and put are O(1); Python's OrderedDict and functools.lru_cache give it for free, and Redis uses a sampled approximation.

1 Something has to go

Your cache has room for one million entries, and the millionth-and-first arrives. Something has to go. Which one? The most common answer, in interviews and in production, is to evict the entry that was used **least recently**: **LRU**. And yes, you may be asked to build one on a whiteboard.

2 The idea

Entries used recently are likely to be used again soon (temporal locality). So keep the recent ones, and when the cache is full, evict the one that hasn't been touched for the longest time. Every read *or* write of a key makes it the most recent.

3 O(1) get and put: hash map + doubly linked list

The interview challenge is doing both operations in constant time. A **hash map** gives O(1) lookup by key but no order. A **list** gives order, but finding a key is O(n). Combine them: a hash map from key to a **node** in a **doubly linked list**, ordered from most recent at the **head** to least recent at the **tail**. Doubly linked, because unlinking a node needs its previous neighbour, and we want that in O(1) too.

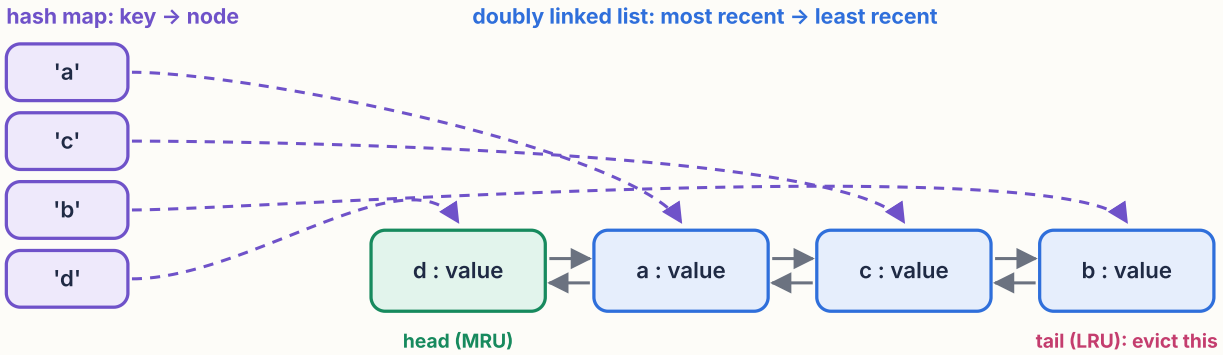


Figure 1. The map finds a node in O(1); the list keeps recency order, so the tail is always the eviction candidate.

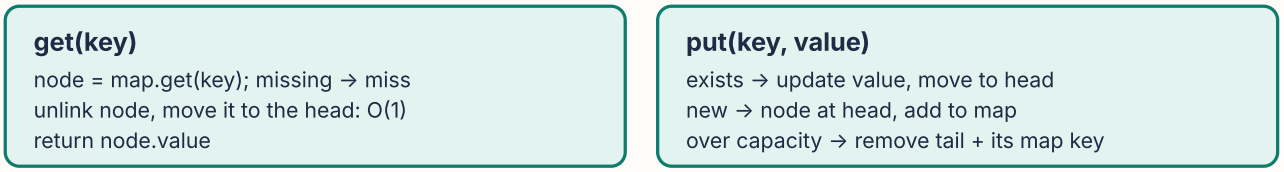


Figure 2. Both operations touch a constant number of pointers. Dummy head and tail nodes avoid special cases for empty lists.

4 In real code

In Python you rarely write the linked list yourself. `OrderedDict` keeps order and has `move_to_end` and `popitem(last=False)`, which is all you need. And `@functools.lru_cache(maxsize=1024)` wraps a function with a ready-made LRU cache. Java's `LinkedHashMap` with access order and `removeEldestEntry` does the same.

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return None
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

Figure 3. A complete LRU cache with OrderedDict. In an interview, be ready to also write the hash map + linked list version.

WORKED EXAMPLE: AN API GATEWAY RESPONSE CACHE

An API gateway caches responses for frequently requested endpoints, like the product catalogue for a category. It keeps the **10,000 most recently requested** responses in an in-memory LRU cache. Popular categories stay cached because they're requested constantly; rarely viewed ones get evicted as new requests arrive. Memory stays bounded at 10,000 entries no matter how many categories exist.

5 LRU in Redis, and LRU vs LFU

Redis doesn't keep a perfect LRU list: that would cost memory and work on every key. With `maxmemory-policy allkeys-lru`, it **samples** a few keys (`maxmemory-samples`, 5 by default) and evicts the least recently used among them. This **approximated LRU** is close enough in practice, and raising the sample size to 10 makes it closer.

When would you choose LRU over LFU? When **recent access predicts future access**, which is true for most workloads: sessions, trending content, recently edited documents. LFU wins when popularity is stable over long periods (next lesson). A known weakness of LRU: one big **scan** (a report reading every product once) pushes the whole hot set out.

COMMON MISTAKES

- Forgetting to move a key to the head on *get*, not just on put.
- Removing the tail node but not its key from the map (a memory leak and wrong lookups).
- Using `lru_cache` on methods with `self`, which keeps every instance alive.
- No `maxmemory` policy in Redis, so writes fail with OOM errors instead of evicting.

6 Key terms

LRU: evict the least recently used entry

Doubly linked list: each node links to previous and next

lru_cache: Python decorator memoising a function with LRU

Scan pollution: a one-off full scan evicting the hot set

MRU end / head: where the most recent entry sits

OrderedDict: Python dict that keeps and reorders insertion order

Approximated LRU: Redis samples a few keys and evicts the oldest

7 Interview questions

Q1. Implement an LRU cache. BASIC

Use a hash map from key to node plus a doubly linked list ordered by recency: on get, move the node to the head; on put, insert at the head and evict the tail when over capacity, all in O(1).

Q2. When would you choose LRU over LFU? BASIC

When recent use predicts the next use, like sessions and trending content, while LFU fits long-term stable popularity.

Q3. Why a doubly linked list rather than a singly linked one? BASIC

To unlink a node in O(1) you need its previous node; a doubly linked list stores it, so moving any node to the head is constant time.

Q4. Redis is full and new writes fail. What do you set? SCENARIO

A maxmemory limit with an eviction policy such as allkeys-lru (or volatile-lru to evict only keys with TTLs), and alert on the eviction rate so you know when to add memory.

Q5. A nightly report reads every product once and the hit ratio collapses the next morning. Why? TRICKY

The scan made every product 'recent' and evicted the real hot set. Use a scan-resistant policy (LFU, or segmented LRU / W-TinyLFU as in Caffeine), or bypass the cache for batch reads.

Q6. How does Redis approximate LRU, and why? TRICKY

It stores a small last-access clock per key and, on eviction, samples a few keys (5 by default) and evicts the oldest of them; a true LRU list would need extra pointers and updates on every access.

Next: SD-F-007 · LFU and FIFO Eviction backendroadmap.com